

SESSION 7 — Subplots & Multi-Panel Figures

Why Subplots Are Needed

In real-world analysis, we often need to:

- Compare multiple charts together
 - Show different perspectives of same data
 - Build dashboards
-

What is a Subplot?

A subplot allows multiple plots inside a single figure.

Two Methods

1. `plt.subplot()` → older method
 2. `plt.subplots()` → modern and preferred
-

Key Concept

Figure → entire canvas

Axes → individual plot areas

✓ `plt.subplot()` (Basic Method)

Syntax

```
plt.subplot(nrows, ncols, index)
```

Parameters

`nrows` → number of rows

`ncols` → number of columns

`index` → position of current plot

Example

`plt.subplot(2,2,1)`

Means:

- 2 rows
- 2 columns
- 1st position

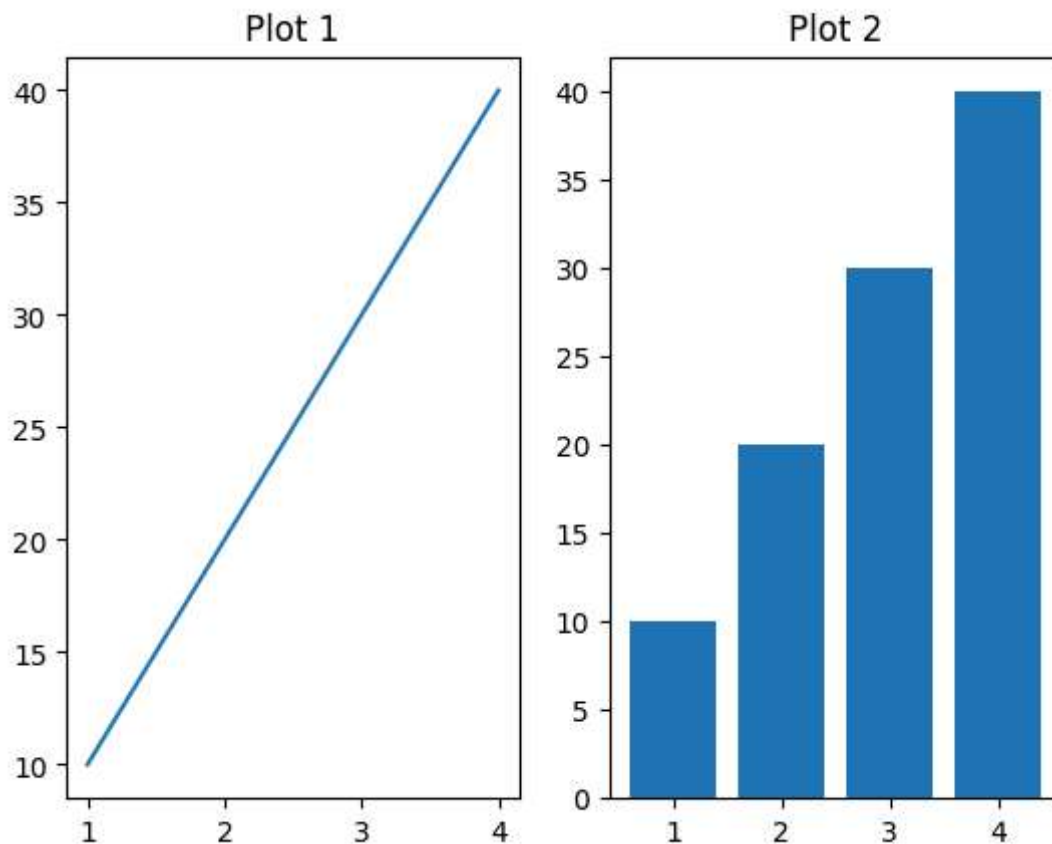
```
import matplotlib.pyplot as plt

x = [1,2,3,4]
y = [10,20,30,40]

# First subplot
plt.subplot(1,2,1)
plt.plot(x, y)
plt.title("Plot 1")

# Second subplot
plt.subplot(1,2,2)
plt.bar(x, y)
plt.title("Plot 2")

plt.show()
```

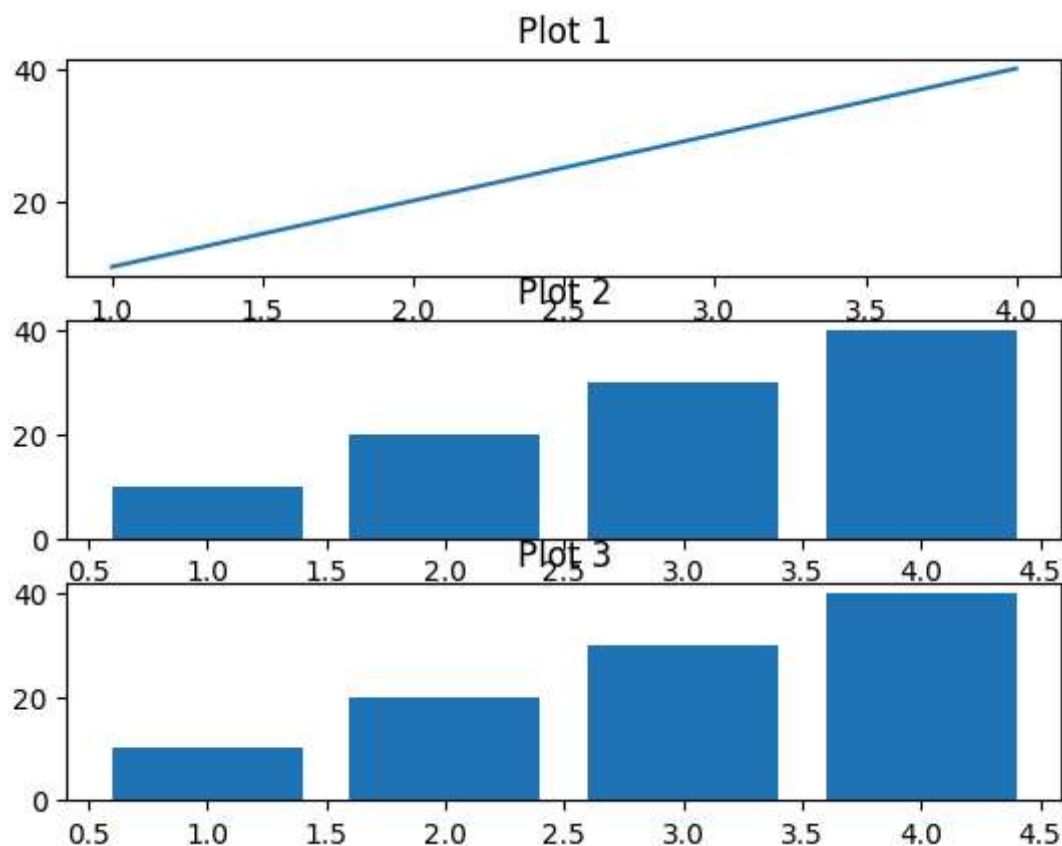


```
import matplotlib.pyplot as plt
```

```
x = [1,2,3,4]  
y = [10,20,30,40]
```

```
# First subplot  
plt.subplot(3,1,1)  
plt.plot(x, y)  
plt.title("Plot 1")  
# Second subplot  
plt.subplot(3,1,2)  
plt.bar(x, y)  
plt.title("Plot 2")  
# third subplot  
plt.subplot(3,1,3)  
plt.bar(x, y)  
plt.title("Plot 3")
```

```
plt.show()
```



✓ plt.subplots() (Preferred Method)

Syntax

```
fig, ax = plt.subplots(nrows=1, ncols=1)
```

Parameters

nrows (default = 1)

ncols (default = 1)

Output

fig → figure object

ax → axes object(s)

Why Preferred?

- Cleaner syntax
- Better control

- Easier for multiple plots

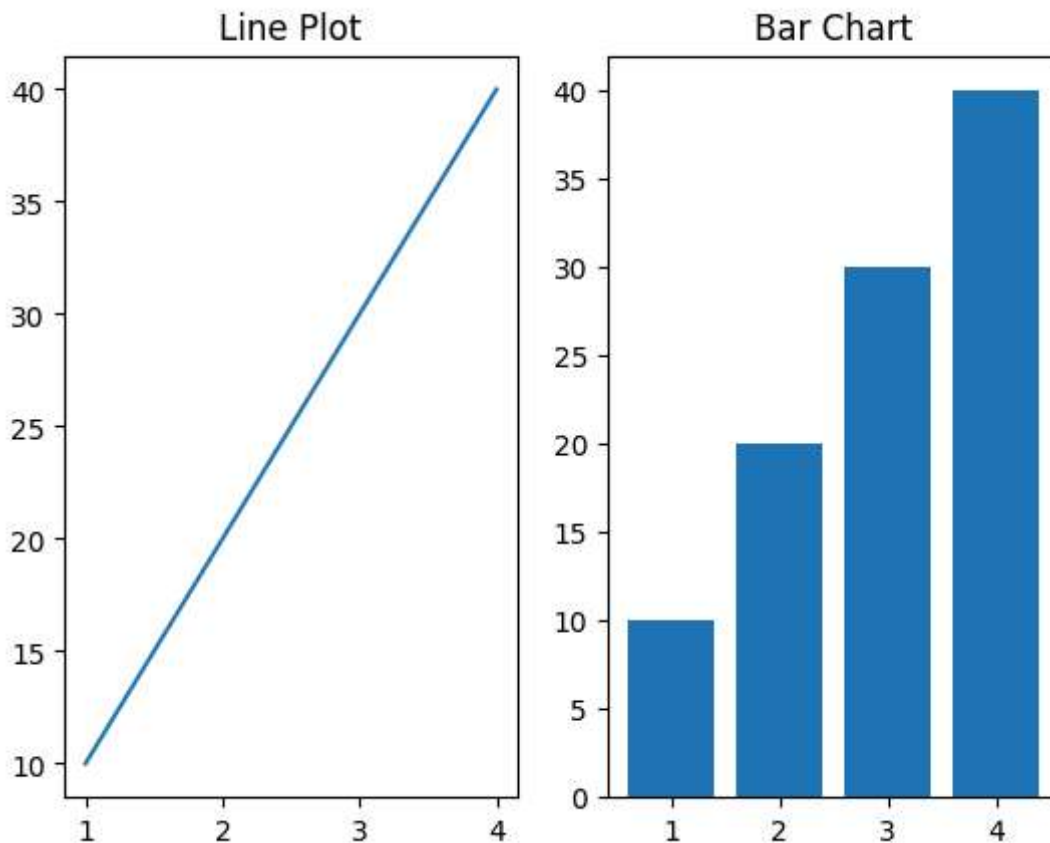
```
# Create figure with 1 row and 2 columns
# fig, ax = plt.subplots(nrows=1, ncols=2)
fig, ax = plt.subplots(nrows=1, ncols=2)

# ax is an array of axes → ax[0], ax[1]

# First plot
ax[0].plot(x, y)
ax[0].set_title("Line Plot")

# Second plot
ax[1].bar(x, y)
ax[1].set_title("Bar Chart")

plt.show()
```



✓ Shared Axes

Purpose

Ensure same scale across plots

Syntax

```
plt.subplots(nrows, ncols, sharex=True, sharey=True)
```

Parameters

sharex (default=False)

sharey (default=False)

Use Case

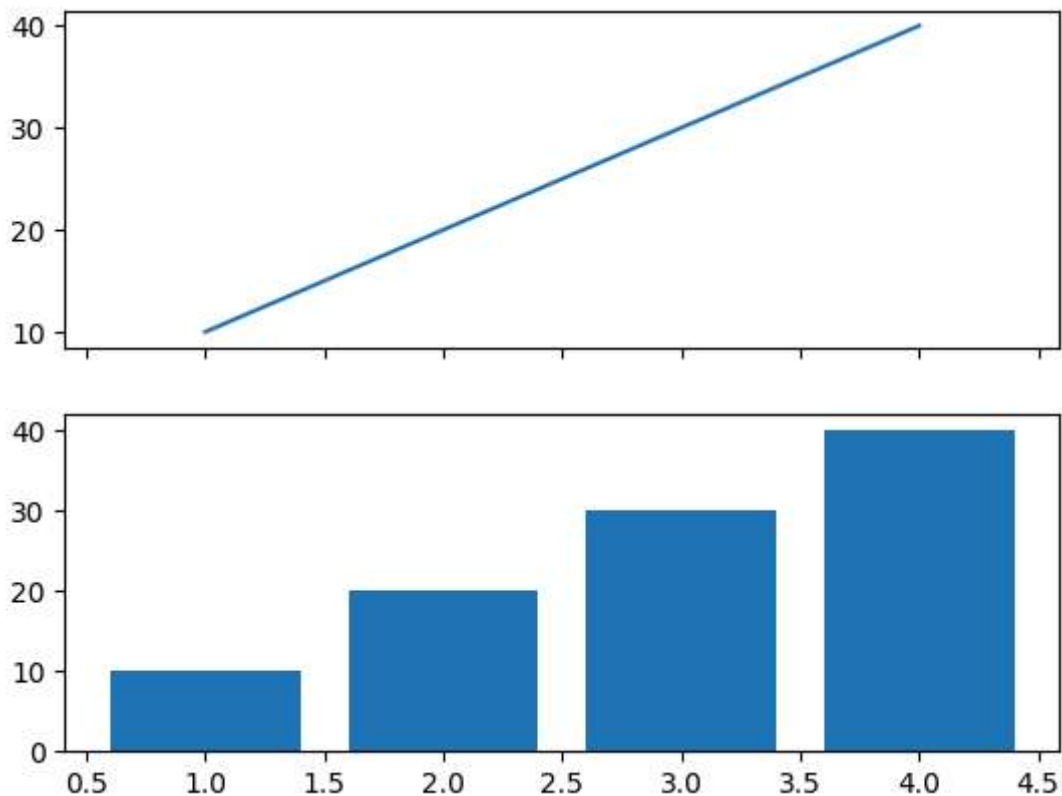
- Comparing trends
- Maintaining consistency

```
fig, ax = plt.subplots(2, 1, sharex=True)
```

```
ax[0].plot(x, y)
```

```
ax[1].bar(x, y)
```

```
plt.show()
```



✓ Grid Layout

You can arrange plots in grid format:

Example:

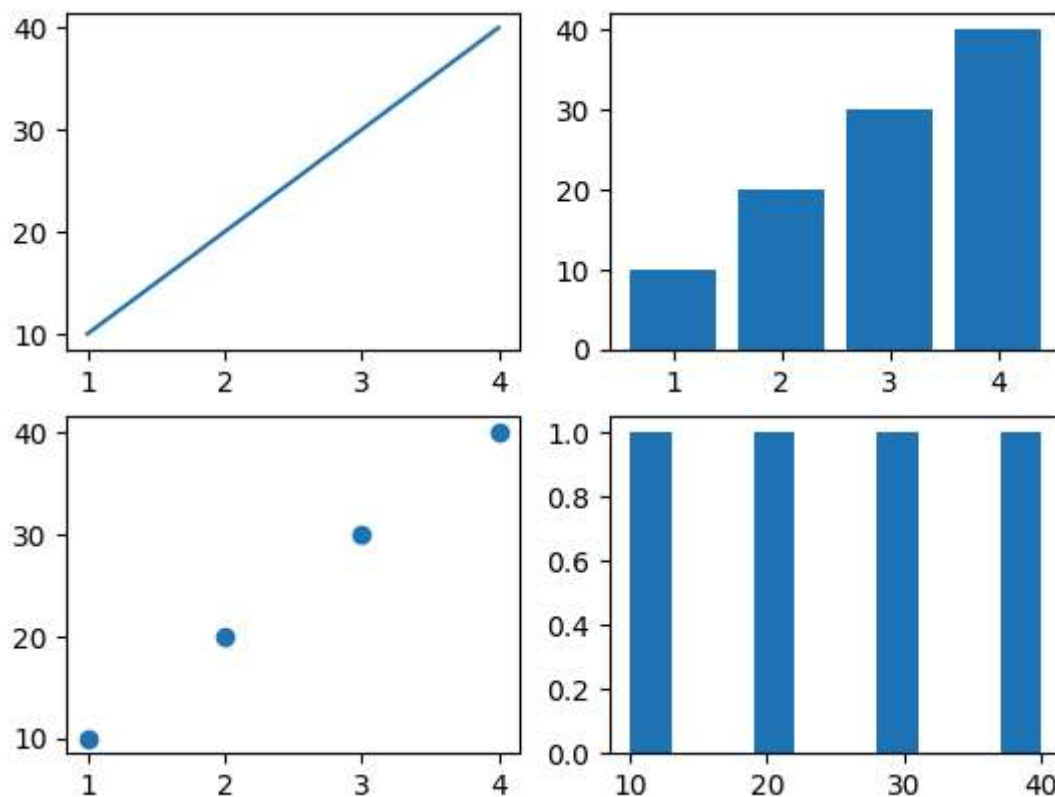
2 rows × 2 columns

```
[ (0,0), (0,1) ]  
[ (1,0), (1,1) ]
```

```
fig, ax = plt.subplots(2,2)
```

```
ax[0,0].plot(x,y)  
ax[0,1].bar(x,y)  
ax[1,0].scatter(x,y)  
ax[1,1].hist(y)
```

```
plt.show()
```



SESSION 8 — Charts Customization & Styling

Why Customization Matters

Default charts are functional but not always clear.

Customization improves:

- Readability
 - Presentation
 - Professional appearance
-

Key Areas

- Figure size
- Line styles
- Colors
- Themes

✓ Figure Size

Syntax

```
plt.figure(figsize=(width, height))
```

Parameters

figsize → tuple (width, height) in inches

Default → depends on environment

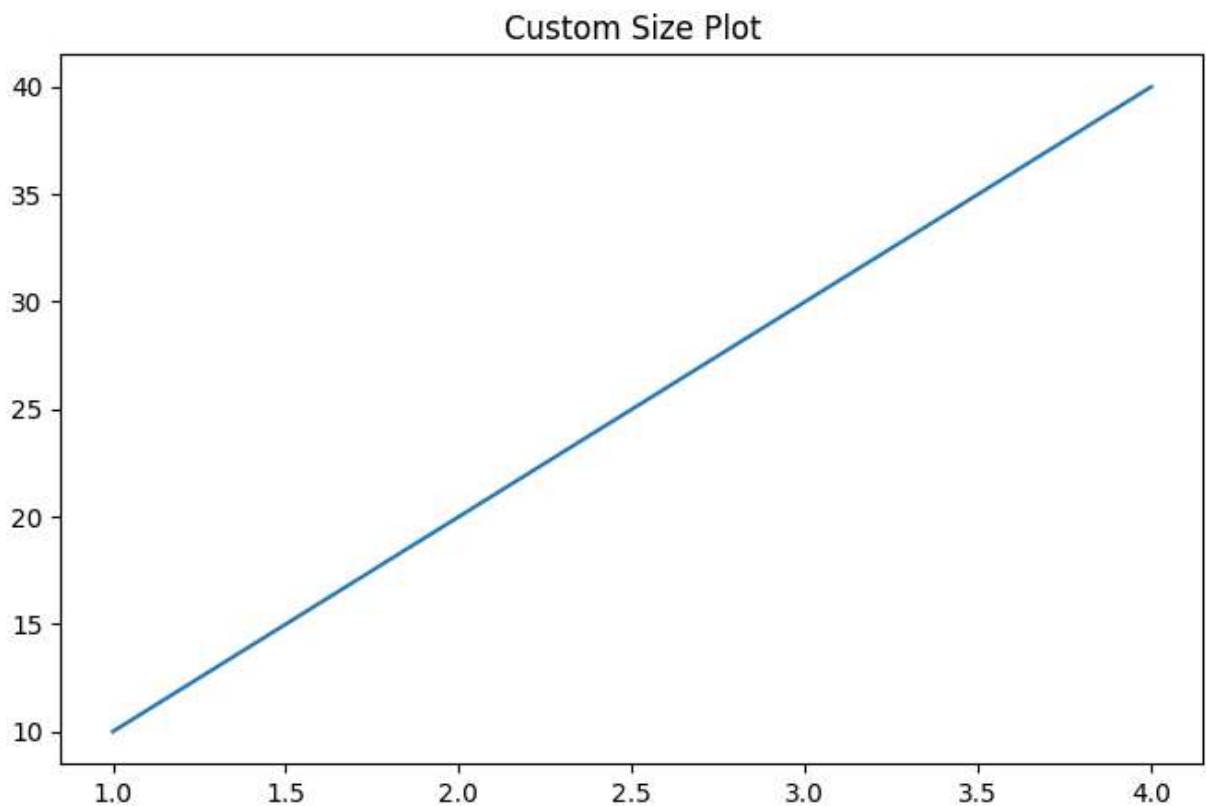
Use Case

- Larger plots for presentations

```
plt.figure(figsize=(8,5))

plt.plot(x, y)

plt.title("Custom Size Plot")
plt.show()
```

✓ Line Styles

Syntax (in plt.plot)

linestyle = '-'

Options

'-' → solid (default)

'--' → dashed

'.' → dotted

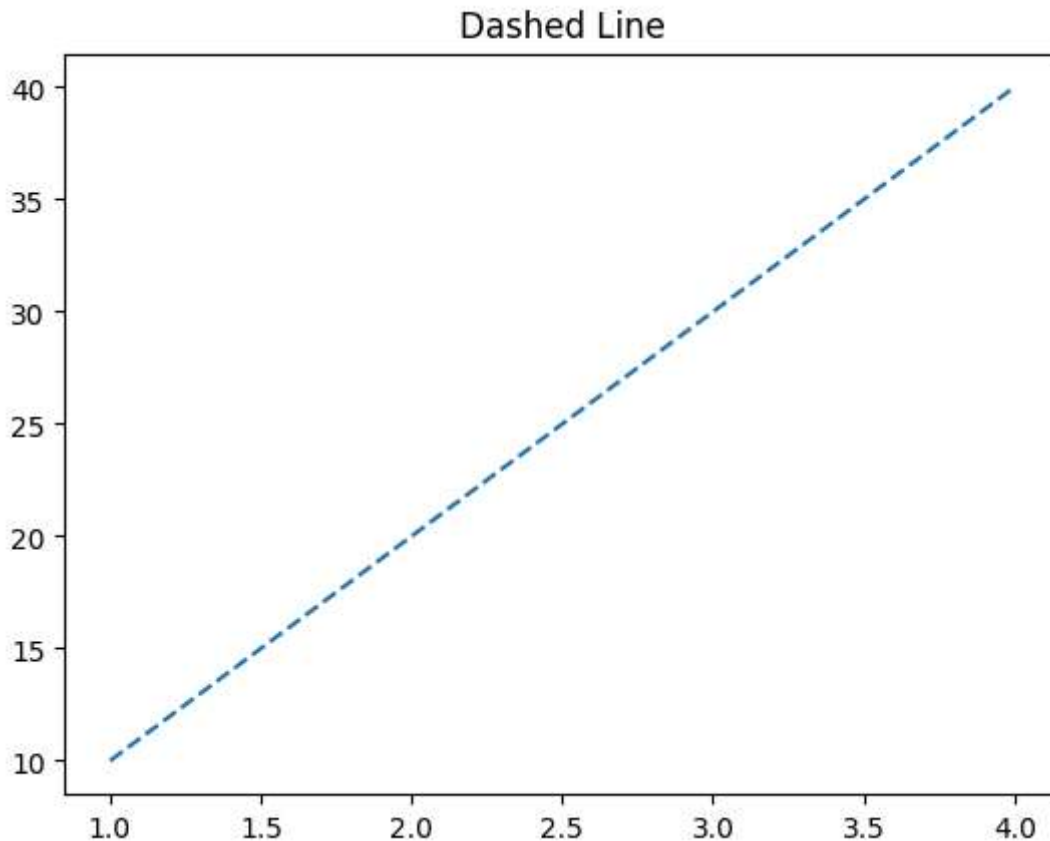
'-.' → dash-dot

Use Case

- Differentiate multiple lines

```
plt.plot(x, y, linestyle='--')
```

```
plt.title("Dashed Line")  
plt.show()
```



✓ Themes using plt.style.use()

Syntax

```
plt.style.use("style_name")
```

Common Styles

- 'ggplot'
- 'seaborn'
- 'dark_background'
- 'classic'

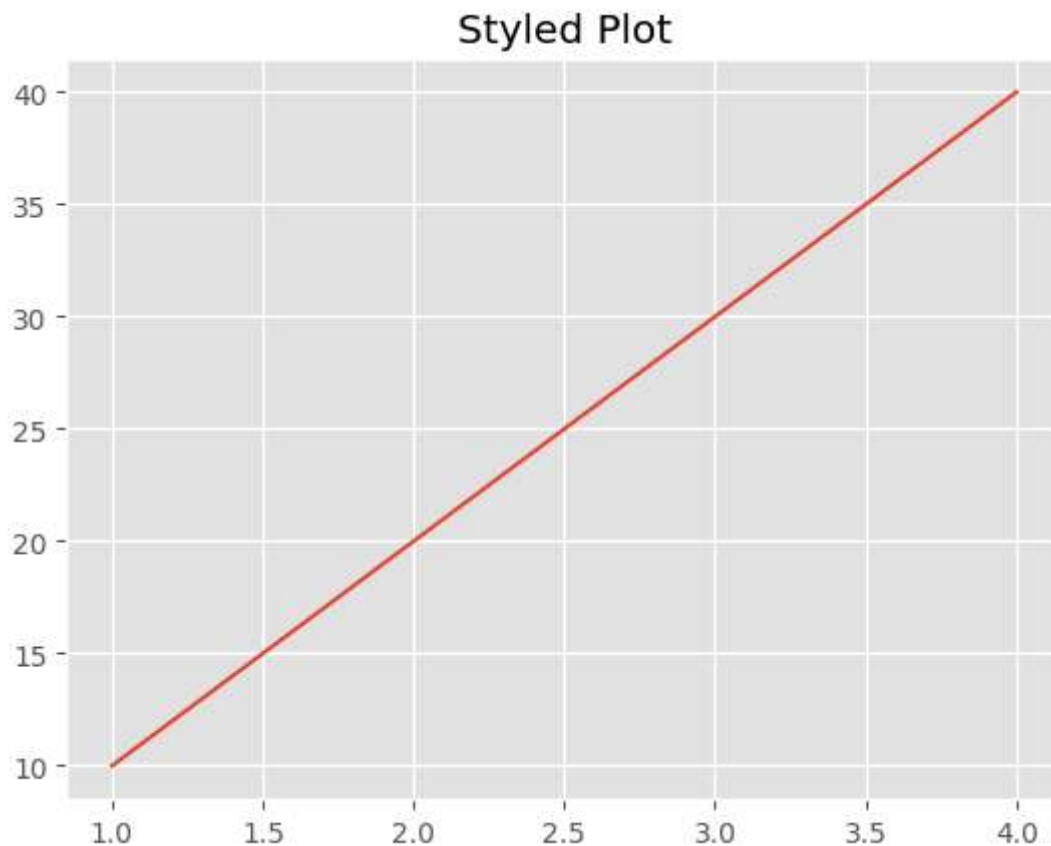
Use Case

- Apply consistent design

```
plt.style.use('ggplot')

plt.plot(x, y)

plt.title("Styled Plot")
plt.show()
```



✓ Colors and Palettes

Ways to Define Colors

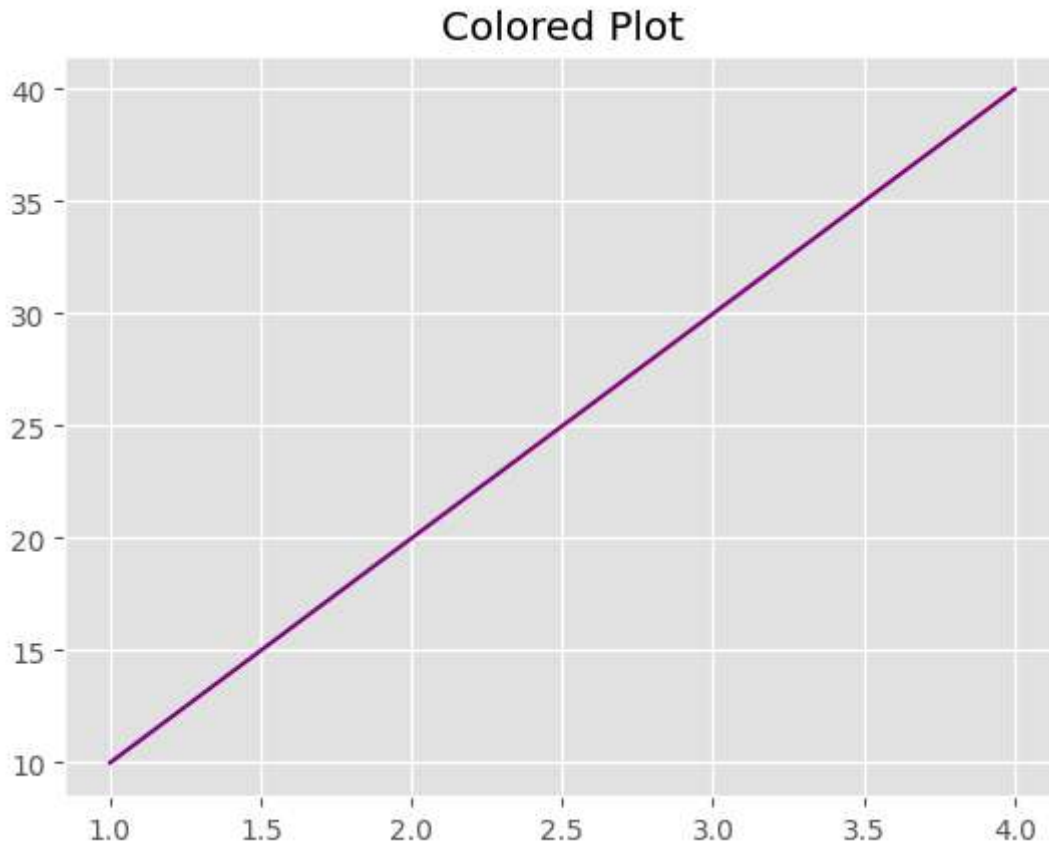
- Named colors ('red', 'blue')
- Hex codes (#FF5733)

Use Case

- Improve clarity
- Highlight key data

```
plt.plot(x, y, color='purple')
```

```
plt.title("Colored Plot")  
plt.show()
```



```
plt.style.use('ggplot')  
  
months = ["Jan", "Feb", "Mar", "Apr", "May"]  
sales = [100, 200, 150, 300, 250]  
  
plt.figure(figsize=(8,5))  
  
plt.plot(  
    months,  
    sales,  
    color='black',  
    linewidth=2,  
    marker='o'  
)  
  
plt.title("Corporate Sales Trend")  
plt.xlabel("Months")  
plt.ylabel("Sales")  
  
plt.show()
```

